

DOCUMENTAÇÃO TÉCNICA

SAFE Sistema

Sistema de Autorização e Fluxo Escolar para gestão
de saídas antecipadas e entradas atrasadas

Versão: 1.0

Tecnologia: Laravel 13 · PHP 8.3 · MySQL 8

Gerado em: 27 de maio de 2026

SAFE — Sistema de Autorização e Fluxo Escolar

Sistema web para gerenciamento de **saídas antecipadas** e **entradas atrasadas** de alunos em ambiente escolar SENAI. Implementa um fluxo de autorização multi-etapas com controle por perfil de acesso: Coordenação → Professor → Portaria.

Índice

1. [Visão Geral](#)
 2. [Tecnologias](#)
 3. [Instalação e Configuração](#)
 4. [Usuários de Teste](#)
 5. [Perfis e Permissões](#)
 6. [Fluxos do Sistema](#)
 7. [Estrutura do Banco de Dados](#)
 8. [Modelos e Relacionamentos](#)
 9. [Rotas e Endpoints](#)
 10. [Políticas de Autorização](#)
 11. [Validações dos Formulários](#)
 12. [Eventos e Notificações](#)
 13. [Auditoria](#)
 14. [Dashboards](#)
 15. [Estrutura de Diretórios](#)
 16. [Regras de Negócio](#)
-

1. Visão Geral

O SAFE resolve o controle de fluxo de alunos em escola técnica. Toda saída ou entrada fora do horário regular exige autorização formal e confirmação em múltiplos pontos, gerando rastreabilidade completa com registro de data, hora, responsável e motivo em cada etapa.

Problemas que o sistema resolve

PROBLEMA	SOLUÇÃO
Aluno sai sem autorização formal	Coordenação cria registro obrigatório antes da saída
Professor não sabe se aluno saiu oficialmente	Confirmação do professor é etapa do fluxo
Portaria libera sem saber se há autorização	Portaria só vê liberações autorizadas pelo professor
Aluno chega atrasado sem controle	Coordenação registra e professor confirma presença
Falta de rastreabilidade	Log de auditoria em toda ação do sistema

2. Tecnologias

CAMADA	TECNOLOGIA	VERSÃO
Backend	PHP	^8.3
Framework	Laravel	13.8
Autenticação	Laravel Breeze	—
Frontend	Tailwind CSS	3.x
Interatividade	Alpine.js	3.4.2
Build	Vite	8.0
HTTP Client	Axios	1.16.1
Banco de Dados	MySQL	8.0
Fuso Horário	America/Sao_Paulo	UTC-3
Idioma	Português Brasil	pt_BR

3. Instalação e Configuração

Pré-requisitos

- PHP 8.3+
- Composer
- Node.js 18+
- MySQL 8.0

Passo a passo

```
# 1. Clonar o repositório
git clone <repositório> safesenai
cd safesenai

# 2. Instalar dependências PHP
composer install

# 3. Instalar dependências JavaScript
npm install

# 4. Criar e configurar o arquivo de ambiente
cp .env.example .env
php artisan key:generate
```

Editar **.env** com as configurações do banco:

```
APP_NAME=SAFE
APP_URL=http://safesenai.test
APP_TIMEZONE=America/Sao_Paulo

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=safesenai
DB_USERNAME=root
DB_PASSWORD=
```

```
# 5. Criar as tabelas e popular com dados de teste
php artisan migrate:fresh --seed

# 6. Compilar os assets de frontend
npm run build

# 7. Iniciar o servidor (desenvolvimento)
php artisan serve
```

Acesso local: **http://localhost:8000** ou **http://safesenai.test** (Laragon)

Servidor de desenvolvimento completo

```
# Inicia servidor, worker de fila, log viewer e Vite simultaneamente  
npm run dev
```

4. Usuários de Teste

Gerados pelo seeder. Para recriar: `php artisan migrate:fresh --seed`

Senha de todos os usuários: **1234**

Coordenação

NOME	E-MAIL	SENHA	PERFIL
Coordenação SENAI	<code>coordenacao@safe.test</code>	1234	Coordenador

Professores

NOME	E-MAIL	SENHA	TURMA VINCULADA
Prof. Carlos Souza	<code>carlos.souza@safe.test</code>	1234	Informática — Manhã
Profa. Ana Lima	<code>ana.lima@safe.test</code>	1234	Administração — Tarde

Portaria

NOME	E-MAIL	SENHA	PERFIL
José Porteiro	<code>portaria@safe.test</code>	1234	Porteiro

Turmas e Alunos

Turma: Informática — Técnico em Informática (Manhã) Professor responsável:
Prof. Carlos Souza

MATRÍCULA	ALUNO
2026001	Lucas Ferreira
2026002	Beatriz Santos
2026003	Gabriel Oliveira
2026004	Mariana Costa
2026005	Rafael Almeida
2026006	Júlia Rodrigues
2026007	Thiago Pereira
2026008	Fernanda Lima
2026009	Diego Nascimento
2026010	Amanda Carvalho

Turma: Administração — Técnico em Administração (Tarde) Professor responsável: Profa. Ana Lima

MATRÍCULA	ALUNO
2026011	Pedro Mendes
2026012	Camila Barbosa
2026013	Rodrigo Martins
2026014	Larissa Cardoso
2026015	Felipe Araújo
2026016	Isabela Rocha
2026017	Vinícius Gomes
2026018	Natália Ribeiro
2026019	Eduardo Sousa
2026020	Carolina Silva

5. Perfis e Permissões

O sistema possui três perfis com responsabilidades distintas e complementares.

Coordenador (**coordinator**)

Acesso total ao sistema. É o ponto de entrada de todos os fluxos.

Gerenciamento:

- Cadastrar, editar e desativar alunos
- Cadastrar e gerenciar turmas
- Cadastrar usuários (professores e porteiros) e vincular professores às turmas

Fluxos:

- Registrar saída antecipada (Etapa 1 do fluxo de saída)
- Registrar entrada atrasada (Etapa 1 do fluxo de entrada)
- Cancelar qualquer autorização pendente

Relatórios:

- Visualizar relatórios filtrando por data, status, aluno e professor
- Consultar o log de auditoria de todas as ações

Dashboard:

- Visão geral do dia: total de saídas, liberadas, aguardando portaria, aguardando professor
- Total de entradas atrasadas e pendentes
- Listagem das últimas 8 saídas e 8 entradas atrasadas do dia
- Atalhos rápidos para registrar nova saída, novo aluno, relatórios

Professor (**teacher**)

Acesso restrito exclusivamente à **sua turma vinculada**.

Fluxos:

- Confirmar saída do aluno em sala (Etapa 2 do fluxo de saída)
- Confirmar presença do aluno atrasado (Etapa 2 do fluxo de entrada)

Visualização:

- Apenas registros de alunos da sua turma
- Lista de liberações e entradas atrasadas (filtradas pela turma)
- Detalhes de alunos

Dashboard:

- Saídas pendentes da sua turma aguardando confirmação
- Entradas atrasadas pendentes da sua turma
- Contadores de confirmações realizadas no dia
- Badge na sidebar indicando quantidade de pendências

Professor sem turma vinculada não visualiza nenhum registro.

Porteiro (`gatekeeper`)

Responsável pela confirmação física da saída do aluno.

Fluxos:

- Confirmar saída física do aluno na portaria (Etapa 3 do fluxo de saída)

Visualização:

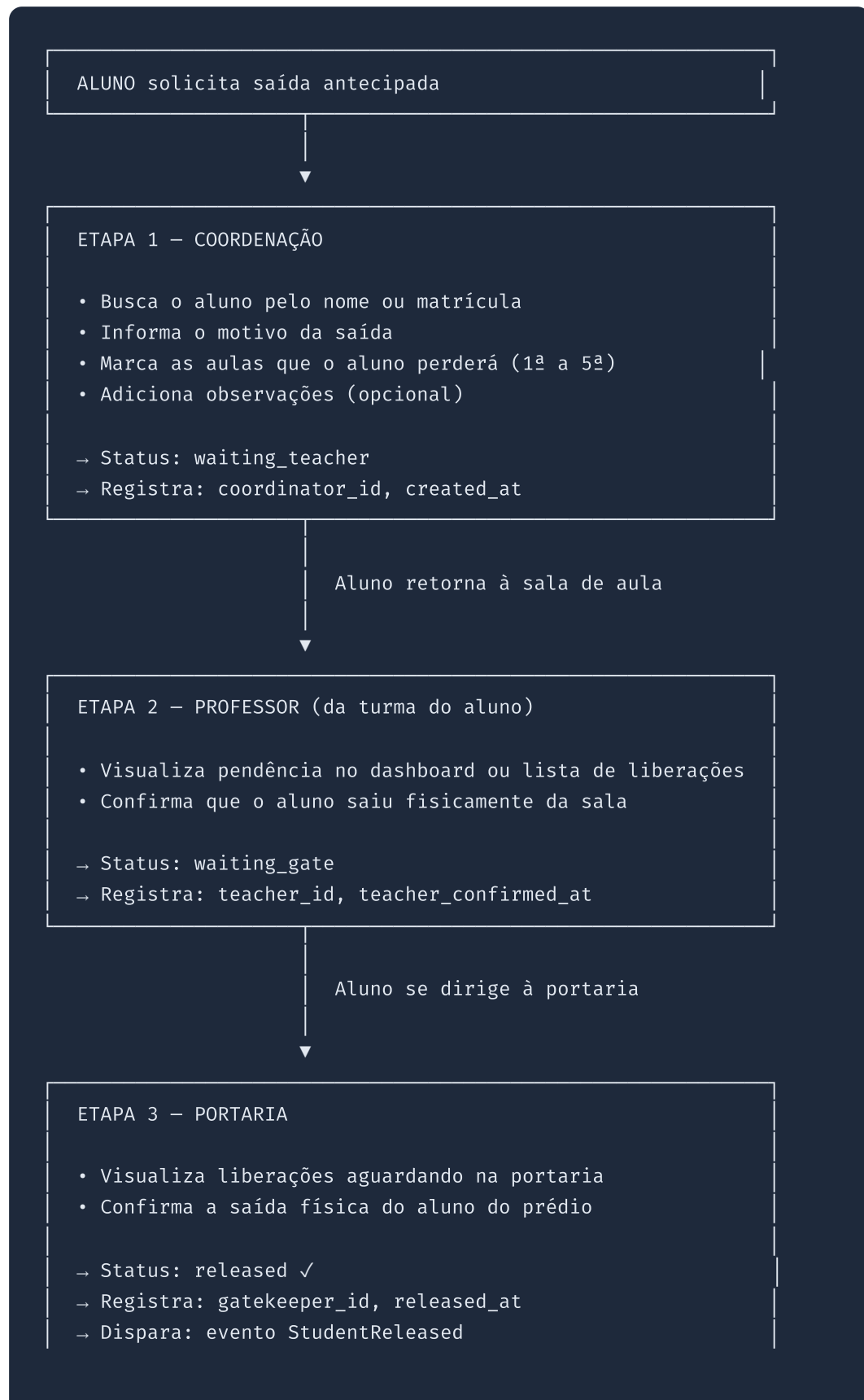
- Todas as autorizações de saída (não visualiza entradas atrasadas)
- Diretório de alunos (apenas leitura)

Dashboard:

- Lista de saídas aguardando confirmação na portaria
- Tabela de alunos já liberados no dia

6. Fluxos do Sistema

6.1 Saída Antecipada — 3 Etapas



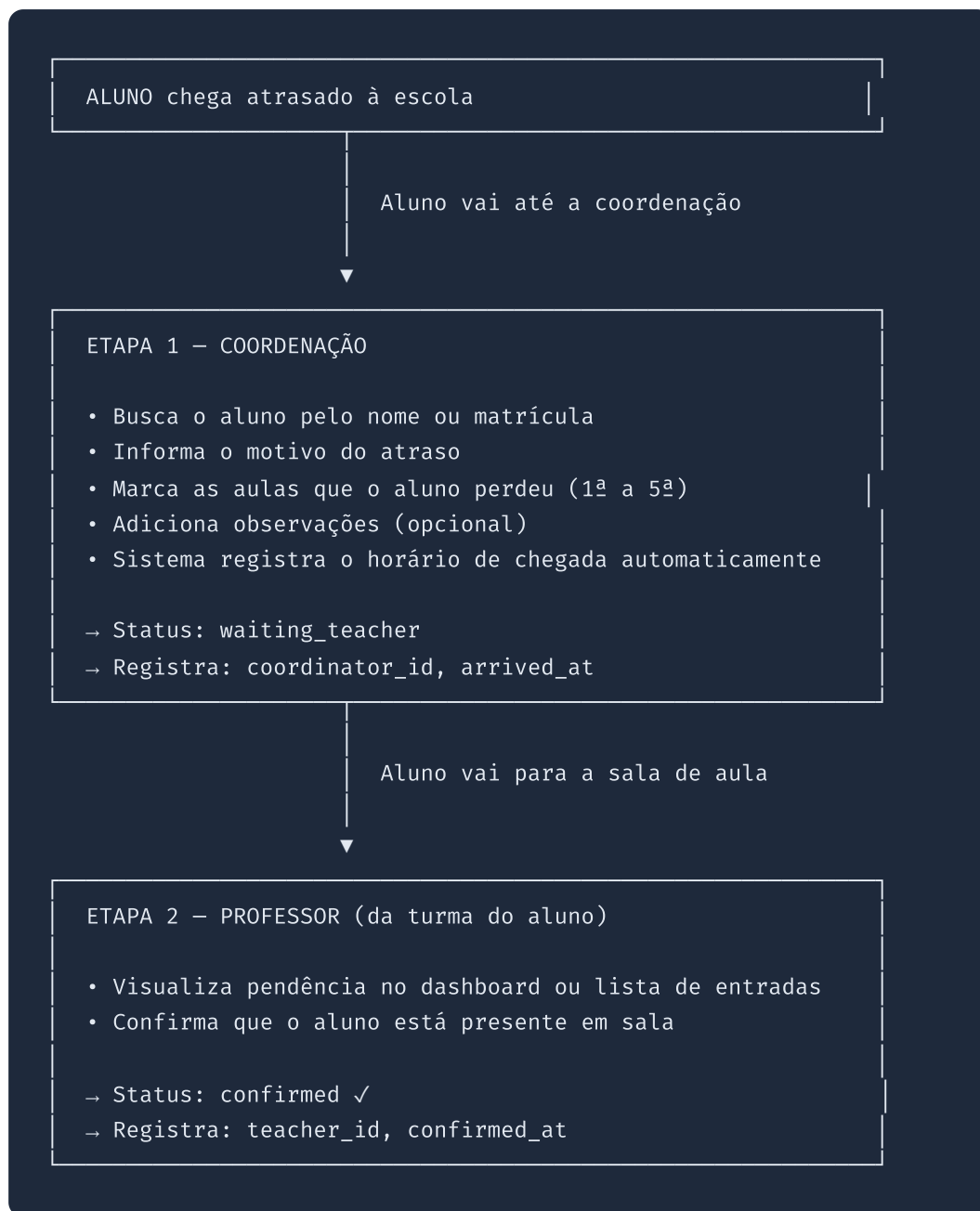
→ Notifica: responsável por e-mail (se cadastrado)

Cancelamento: A coordenação pode cancelar em qualquer etapa (`waiting_teacher` ou `waiting_gate`), mudando o status para `cancelled` .

Transição de status:

```
waiting_teacher → waiting_gate → released
    |
    |_____→ cancelled
```

6.2 Entrada Atrasada — 2 Etapas



Cancelamento: A coordenação pode cancelar enquanto estiver em `waiting_teacher`.

Transição de status:

```
waiting_teacher → confirmed
                  |
                  → cancelled
```

6.3 Aulas Perdidas

Em ambos os fluxos, é possível selecionar múltiplas aulas perdidas pelo aluno (checkboxes):

VALOR	SIGNIFICADO
1	1ª aula / 1ª hora
2	2ª aula / 2ª hora
3	3ª aula / 3ª hora
4	4ª aula / 4ª hora
5	5ª aula / 5ª hora

Os valores são armazenados como array JSON (ex: `[1, 3, 5]`) e exibidos como badges na listagem e na tela de detalhes.

7. Estrutura do Banco de Dados

Diagrama de Relacionamentos

```
classrooms ←—— users (classroom_id, role='teacher')
|
├── students ───→ early_releases ←—— users (coordinator_id)
|                                     ←—— users (teacher_id)
|                                     ←—— users (gatekeeper_id)
|
└── late_entries ←—— users (coordinator_id)
                    ←—— users (teacher_id)

users ───→ audit_logs (user_id)
```

Tabela: `users`

COLUNA	TIPO	NULO	PADRÃO	DESCRIÇÃO
id	bigint PK	não	—	Identificador
name	varchar(255)	não	—	Nome completo
email	varchar(255) unique	não	—	E-mail de login
email_verified_at	timestamp	sim	null	Verificação de e-mail
password	varchar(255)	não	—	Senha (bcrypt)
role	enum	não	<code>teacher</code>	<code>coordinator</code> , <code>teacher</code> , <code>gatekeeper</code>
active	tinyint(1)	não	<code>1</code>	Ativo/Inativo (exclusão lógica)
classroom_id	bigint FK	sim	null	Turma (somente professores)
remember_token	varchar(100)	sim	null	Token de sessão
created_at / updated_at	timestamp	sim	—	Timestamps

FK: `classroom_id` → `classrooms.id` ON DELETE SET NULL

Tabela: **classrooms**

COLUNA	TIPO	NULO	DESCRIÇÃO
id	bigint PK	não	Identificador
name	varchar(255)	não	Nome da turma/curso
grade	varchar(255)	não	Nível (ex: "Técnico em Informática")
shift	varchar(255)	não	Turno: morning , afternoon , evening
year	year(4)	não	Ano letivo
active	tinyint(1)	não	Ativo/Inativo
created_at / updated_at	timestamp	sim	Timestamps

Tabela: **students**

COLUNA	TIPO	NULO	DESCRIÇÃO
id	bigint PK	não	Identificador
classroom_id	bigint FK	não	Turma do aluno
name	varchar(255)	não	Nome completo
registration	varchar(255) unique	não	Número de matrícula
birth_date	date	sim	Data de nascimento
guardian_name	varchar(255)	sim	Nome do responsável
guardian_phone	varchar(255)	sim	Telefone do responsável
guardian_email	varchar(255)	sim	E-mail do responsável (usado para notificações)
active	tinyint(1)	não	Ativo/Inativo (exclusão lógica)
created_at / updated_at	timestamp	sim	Timestamps

FK: `classroom_id` → `classrooms.id` ON DELETE RESTRICT

Tabela: `early_releases`

COLUNA	TIPO	NULO	DESCRIÇÃO
<code>id</code>	<code>bigint PK</code>	não	Identificador
<code>student_id</code>	<code>bigint FK</code>	não	Aluno
<code>coordinator_id</code>	<code>bigint FK</code>	sim	Coordenador que registrou
<code>teacher_id</code>	<code>bigint FK</code>	sim	Professor que confirmou a saída da sala
<code>gatekeeper_id</code>	<code>bigint FK</code>	sim	Porteiro que confirmou a saída do prédio
<code>reason</code>	<code>varchar(255)</code>	não	Motivo da saída
<code>observation</code>	<code>text</code>	sim	Observações adicionais
<code>status</code>	<code>enum</code>	não	<code>waiting_teacher</code> , <code>waiting_gate</code> , <code>released</code> , <code>cancelled</code>
<code>missed_periods</code>	<code>json</code>	sim	Array de aulas perdidas, ex: <code>[1, 2]</code>
<code>teacher_confirmed_at</code>	<code>timestamp</code>	sim	Data/hora da confirmação do professor
<code>released_at</code>	<code>timestamp</code>	sim	Data/hora da liberação na portaria
<code>created_at</code> / <code>updated_at</code>	<code>timestamp</code>	sim	Timestamps

FKs: `student_id` , `coordinator_id` , `teacher_id` , `gatekeeper_id` → `users.id`
/ `students.id` ON DELETE RESTRICT

Tabela: `late_entries`

COLUNA	TIPO	NULO	DESCRIÇÃO
id	bigint PK	não	Identificador
student_id	bigint FK	não	Aluno
coordinator_id	bigint FK	não	Coordenador que registrou
teacher_id	bigint FK	sim	Professor que confirmou a presença
reason	varchar(255)	não	Motivo do atraso
observation	text	sim	Observações adicionais
status	enum	não	<code>waiting_teacher</code> , <code>confirmed</code> , <code>cancelled</code>
missed_periods	json	sim	Array de aulas perdidas, ex: <code>[1]</code>
arrived_at	timestamp	sim	Horário de chegada registrado automaticamente
confirmed_at	timestamp	sim	Data/hora da confirmação do professor
created_at / updated_at	timestamp	sim	Timestamps

Tabela: `audit_logs`

COLUNA	TIPO	NULO	DESCRIÇÃO
id	bigint PK	não	Identificador
user_id	bigint FK	sim	Usuário que executou a ação
action	varchar(255)	não	Identificador da ação (ex: <code>early_release.created</code>)
model_type	varchar(255)	sim	Classe do modelo afetado
model_id	bigint	sim	ID do registro afetado
old_values	json	sim	Estado anterior do registro
new_values	json	sim	Estado novo do registro
ip_address	varchar(45)	sim	Endereço IP do usuário
user_agent	varchar(255)	sim	Navegador/dispositivo
created_at / updated_at	timestamp	sim	Timestamps

FK: `user_id` → `users.id` ON DELETE SET NULL

8. Modelos e Relacionamentos

User

```
// Relacionamentos
classroom()          // BelongsTo Classroom      - turma vinculada
releasesAsCoordinator() // HasMany EarlyRelease (coordinator_id)
releasesAsTeacher()   // HasMany EarlyRelease (teacher_id)
releasesAsGatekeeper() // HasMany EarlyRelease (gatekeeper_id)
auditLogs()           // HasMany AuditLog

// Métodos
isCoordinator() // role === 'coordinator'
isTeacher()     // role === 'teacher'
isGatekeeper()  // role === 'gatekeeper'

// Acessores
$user->role_label // 'Coordenação' | 'Professor' | 'Porteiro' | 'Desconhecido'
```

Classroom

```
// Relacionamentos
teacher() // HasOne User (where role='teacher') - professor responsável
students() // HasMany Student

// Acessores
$classroom->shift_label // 'Manhã' | 'Tarde' | 'Noite'
$classroom->full_name    // "Informática - Técnico em Informática (Manhã)"
```

Student

```
// Relacionamentos
classroom() // BelongsTo Classroom
earlyReleases() // HasMany EarlyRelease
lateEntries() // HasMany LateEntry

// Métodos de verificação
hasPendingRelease() // Verifica status IN ('waiting_teacher', 'waiting_student')
hasPendingLateEntry() // Verifica status = 'waiting_teacher'
```

EarlyRelease

```
// Relacionamentos
student()      // BelongsTo Student
coordinator() // BelongsTo User (coordinator_id)
teacher()      // BelongsTo User (teacher_id)
gatekeeper()  // BelongsTo User (gatekeeper_id)

// Escopos de query
scopeWaitingTeacher($query) // WHERE status = 'waiting_teacher'
scopeWaitingGate($query)    // WHERE status = 'waiting_gate'
scopeReleased($query)       // WHERE status = 'released'
scopeToday($query)          // WHERE DATE(created_at) = hoje

// Acessores
$release->missed_periods_label // "1ª, 2ª, 3ª Aula" ou null
$release->status_label          // 'Aguardando Professor' | 'Aguardando
$release->status_color          // 'blue' | 'yellow' | 'green' | 'red'
```

LateEntry

```
// Relacionamentos
student()      // BelongsTo Student
coordinator() // BelongsTo User (coordinator_id)
teacher()      // BelongsTo User (teacher_id)

// Escopos de query
scopeWaitingTeacher($query) // WHERE status = 'waiting_teacher'
scopeConfirmed($query)      // WHERE status = 'confirmed'
scopeToday($query)          // WHERE DATE(created_at) = hoje

// Acessores
$entry->missed_periods_label // "1ª, 2ª Aula" ou null
$entry->status_label          // 'Aguardando Professor' | 'Confirmado'
$entry->status_color          // 'blue' | 'green' | 'red'
```

AuditLog

```
// Relacionamentos
user() // BelongsTo User

// Método estático para registrar ações
AuditLog::record(
    action: 'early_release.created',
    model: $release,           // opcional
    oldValues: [],             // opcional
    newValues: $data           // opcional
);
// Captura automaticamente: user_id, ip_address, user_agent
```

9. Rotas e Endpoints

Middleware de Perfil

```
role:coordinator           → Apenas coordenadores
role:teacher                → Apenas professores
role:gatekeeper             → Apenas porteiros
role:coordinator,teacher    → Coordenadores e professores
role:coordinator,teacher,gatekeeper → Todos os perfis
```

Dashboard

MÉTODO	URL	PERFIL	DESCRIÇÃO
GET	<code>/dashboard</code>	Todos	Redireciona para dashboard do perfil

Alunos

MÉTODO	URL	PERFIL	DESCRIÇÃO
GET	<code>/students</code>	Todos	Listagem com filtros
GET	<code>/students/create</code>	Coordenação	Formulário de cadastro
POST	<code>/students</code>	Coordenação	Salvar novo aluno
GET	<code>/students/{id}</code>	Todos	Detalhes do aluno + histórico
GET	<code>/students/{id}/edit</code>	Coordenação	Formulário de edição
PATCH	<code>/students/{id}</code>	Coordenação	Atualizar aluno
DELETE	<code>/students/{id}</code>	Coordenação	Desativar aluno
GET	<code>/api/students/search?q=</code>	Todos	Busca JSON para autocomplete

Turmas

MÉTODO	URL	PERFIL	DESCRIÇÃO
GET	<code>/classrooms</code>	Coordenação	Listagem de turmas
GET	<code>/classrooms/create</code>	Coordenação	Formulário de cadastro
POST	<code>/classrooms</code>	Coordenação	Salvar turma
GET	<code>/classrooms/{id}/edit</code>	Coordenação	Formulário de edição
PATCH	<code>/classrooms/{id}</code>	Coordenação	Atualizar turma
DELETE	<code>/classrooms/{id}</code>	Coordenação	Desativar turma

Usuários

MÉTODO	URL	PERFIL	DESCRIÇÃO
GET	/users	Coordenação	Listagem de usuários
GET	/users/create	Coordenação	Formulário de cadastro
POST	/users	Coordenação	Salvar usuário
GET	/users/{id}/edit	Coordenação	Formulário de edição
PATCH	/users/{id}	Coordenação	Atualizar usuário
DELETE	/users/{id}	Coordenação	Desativar usuário

Saídas Antecipadas

MÉTODO	URL	PERFIL	ETAPA	DESCRIÇÃO
GET	/early-releases	Todos	—	Listagem com filtros
GET	/early-releases/create	Coordenação	—	Formulário de registro
POST	/early-releases	Coordenação	Etapa 1	Registrar saída
GET	/early-releases/{id}	Todos	—	Detalhes + linha do tempo
POST	/early-releases/{id}/confirm-teacher	Professor	Etapa 2	Confirmar saída da sala
POST	/early-releases/{id}/confirm	Portaria	Etapa 3	Confirmar saída do prédio
POST	/early-releases/{id}/cancel	Coordenação	—	Cancelar autorização

Entradas Atrasadas

MÉTODO	URL	PERFIL	ETAPA	DESCRIÇÃO
GET	<code>/late-entries</code>	Coord. + Prof.	—	Listagem com filtros
GET	<code>/late-entries/create</code>	Coordenação	—	Formulário de registro
POST	<code>/late-entries</code>	Coordenação	Etapa 1	Registrar entrada
GET	<code>/late-entries/{id}</code>	Coord. + Prof.	—	Detalhes + linha do tempo
POST	<code>/late-entries/{id}/confirm</code>	Professor	Etapa 2	Confirmar presença
POST	<code>/late-entries/{id}/cancel</code>	Coordenação	—	Cancelar

Relatórios e Auditoria

MÉTODO	URL	PERFIL	DESCRIÇÃO
GET	<code>/reports</code>	Coordenação	Relatório de saídas com filtros
GET	<code>/audit-logs</code>	Coordenação	Log de auditoria de ações

10. Políticas de Autorização

EarlyReleasePolicy

MÉTODO	COORDENAÇÃO	PROFESSOR	PORTARIA	CONDIÇÃO EXTRA
<code>viewAny</code>	✓	✓	✓	—
<code>view</code>	✓	✓	✓	Professor: classroom_id do professor = classroom_id do aluno
<code>create</code>	✓	✗	✗	—
<code>confirmTeacher</code>	✗	✓	✗	Status = <code>waiting_teacher</code> + classroom do professor = classroom do aluno
<code>confirm</code>	✗	✗	✓	Status = <code>waiting_gate</code>
<code>cancel</code>	✓	✗	✗	Status IN (<code>waiting_teacher</code> , <code>waiting_gate</code>)

LateEntryPolicy

MÉTODO	COORDENAÇÃO	PROFESSOR	PORTARIA	CONDIÇÃO EXTRA
<code>viewAny</code>	✓	✓	✗	—
<code>view</code>	✓	✓	✗	Professor: classroom do professor = classroom do aluno
<code>create</code>	✓	✗	✗	—
<code>confirm</code>	✗	✓	✗	Status = <code>waiting_teacher</code> + classroom match
<code>cancel</code>	✓	✗	✗	Status = <code>waiting_teacher</code>

StudentPolicy

MÉTODO	COORDENAÇÃO	PROFESSOR	PORTARIA
<code>viewAny</code>	✓	✓	✓
<code>view</code>	✓	✓	✓
<code>create</code>	✓	✗	✗
<code>update</code>	✓	✗	✗
<code>delete</code>	✓	✗	✗

11. Validações dos Formulários

Registrar Saída Antecipada (`StoreEarlyReleaseRequest`)

CAMPO	REGRAS	MENSAGEM DE ERRO
<code>student_id</code>	obrigatório, existe em <code>students.id</code>	"O campo aluno é obrigatório."
<code>reason</code>	obrigatório, string, máx. 255 chars	"O campo motivo é obrigatório."
<code>observation</code>	opcional, string, máx. 1000 chars	—
<code>missed_periods</code>	obrigatório, array, mínimo 1 item	"Selecione ao menos uma aula perdida."
<code>missed_periods.*</code>	inteiro, entre 1 e 5	"Período inválido."

Autorização: somente `coordinator` .

Registrar Entrada Atrasada (`StoreLateEntryRequest`)

Mesmas regras de `StoreEarlyReleaseRequest` .

Cadastrar Usuário (`StoreUserRequest`)

CAMPO	REGRAS
<code>name</code>	obrigatório, string, máx. 255
<code>email</code>	obrigatório, e-mail válido, único em <code>users</code>
<code>role</code>	obrigatório, em (<code>teacher</code> , <code>gatekeeper</code>)
<code>classroom_id</code>	opcional, existe em <code>classrooms.id</code>
<code>password</code>	obrigatório, mínimo 8 caracteres, confirmado

Cadastrar Aluno (`StoreStudentRequest`)

CAMPO	REGRAS
<code>classroom_id</code>	obrigatório, existe em <code>classrooms.id</code>
<code>name</code>	obrigatório, string, máx. 255
<code>registration</code>	obrigatório, string, máx. 50, único em <code>students</code>
<code>birth_date</code>	opcional, data válida
<code>guardian_name</code>	opcional, string, máx. 255
<code>guardian_phone</code>	opcional, string, máx. 20
<code>guardian_email</code>	opcional, e-mail válido, máx. 255

Cadastrar Turma (`StoreClassroomRequest`)

CAMPO	REGRAS
<code>name</code>	obrigatório, string, máx. 100
<code>grade</code>	obrigatório, string, máx. 50
<code>shift</code>	obrigatório, em (<code>morning</code> , <code>afternoon</code> , <code>evening</code>)
<code>year</code>	obrigatório, inteiro, entre 2020 e 2099

12. Eventos e Notificações

Evento: `StudentReleased`

Disparado na Etapa 3 (confirmação da portaria). Carrega a instância de `EarlyRelease` com todos os relacionamentos carregados.

Listener: `SendReleaseNotification`

Implementa `ShouldQueue` (execução assíncrona na fila).

Ações do listener:

1. Log de sistema — registra no canal `default` :

```
[INFO] Liberação confirmada: {aluno} ({turma})  
Professor: {professor} | Porteiro: {porteiro}  
Motivo: {motivo} | Horário: {released_at}
```

2. Simulação de WhatsApp — registra no canal `single` :

```
[WhatsApp] Para: {responsável}  
Seu filho(a) {aluno} foi liberado(a) às {hora}.  
Autorizado por: {professor} (professor)  
Motivo: {motivo}
```

3. E-mail ao responsável — envia `StudentReleasedMail` se o aluno tiver `guardian_email` cadastrado.

13. Auditoria

Toda ação relevante no sistema é registrada automaticamente via

```
AuditLog::record() .
```

Ações auditadas

AÇÃO	QUANDO
<code>user.created</code>	Novo usuário cadastrado
<code>user.updated</code>	Usuário editado
<code>user.deactivated</code>	Usuário desativado
<code>student.created</code>	Novo aluno cadastrado
<code>student.updated</code>	Aluno editado
<code>student.deactivated</code>	Aluno desativado
<code>classroom.created</code>	Turma criada
<code>classroom.updated</code>	Turma editada
<code>classroom.deactivated</code>	Turma desativada
<code>early_release.created</code>	Saída registrada pela coordenação
<code>early_release.teacher_confirmed</code>	Professor confirmou saída da sala
<code>early_release.confirmed</code>	Portaria confirmou saída do prédio
<code>early_release.cancelled</code>	Saída cancelada
<code>late_entry.created</code>	Entrada atrasada registrada
<code>late_entry.confirmed</code>	Professor confirmou presença
<code>late_entry.cancelled</code>	Entrada cancelada

Dados gravados por registro

- `user_id` — quem executou a ação
- `action` — identificador da ação
- `model_type` + `model_id` — registro afetado
- `old_values` — estado anterior (para atualizações)
- `new_values` — estado novo (para atualizações)
- `ip_address` — endereço IP
- `user_agent` — navegador/dispositivo

14. Dashboards

Coordenação

Total Saídas	Liber. Hoje	Ag. Porta.	Ag. Prof.	Atraso Hoje	Atraso Pend.	Total Alunos
--------------	-------------	------------	-----------	-------------	--------------	--------------

Últimas Saídas
Antecipadas (8)
[lista do dia]

Últimas Entradas
Atrasadas (8)
[lista do dia]

Professor

Saídas Confirmar (sua turma)	Saídas Confirmadas Hoje	Entradas Confirmar (sua turma)	Entradas Confirmadas Hoje
------------------------------	-------------------------	--------------------------------	---------------------------

Saídas Aguardando
Confirmação
[botão Confirmar]

Entradas Aguardando
Confirmação
[botão Confirmar]

Portaria

Aguardando
Portaria
[botão Liberar]

Liberados Hoje
[tabela resumida]

15. Estrutura de Diretórios

```

safesenai/
├── app/
│   ├── Events/
│   │   └── StudentReleased.php          # Evento: saída confirmada na
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── AuditLogController.php  # Log de auditoria
│   │   │   ├── ClassroomController.php # Gerenciar turmas
│   │   │   ├── DashboardController.php # Dashboards por perfil
│   │   │   ├── EarlyReleaseController.php # Saídas antecipadas
│   │   │   ├── LateEntryController.php # Entradas atrasadas
│   │   │   ├── ReportController.php    # Relatórios
│   │   │   ├── StudentController.php   # Gerenciar alunos
│   │   │   └── UserController.php      # Gerenciar usuários
│   │   ├── Middleware/
│   │   │   └── RoleMiddleware.php       # Controle de acesso por perfil
│   │   └── Requests/
│   │       ├── StoreClassroomRequest.php
│   │       ├── StoreEarlyReleaseRequest.php
│   │       ├── StoreLateEntryRequest.php
│   │       ├── StoreStudentRequest.php
│   │       ├── StoreUserRequest.php
│   │       └── UpdateStudentRequest.php
│   ├── Listeners/
│   │   └── SendReleaseNotification.php  # Notifica responsável na libe
│   ├── Mail/
│   │   └── StudentReleasedMail.php      # E-mail de confirmação de saí
│   ├── Models/
│   │   ├── AuditLog.php
│   │   ├── Classroom.php
│   │   ├── EarlyRelease.php
│   │   ├── LateEntry.php
│   │   ├── Student.php
│   │   └── User.php
│   ├── Policies/
│   │   ├── EarlyReleasePolicy.php
│   │   ├── LateEntryPolicy.php
│   │   └── StudentPolicy.php
│   └── Providers/
│       └── AppServiceProvider.php       # Registra policies e event li
├── database/
│   └── migrations/
│       ├── 000000_create_users_table.php
│       ├── 000010_create_classrooms_table.php
│       └── 000020_create_students_table.php

```



```

├── 000030_create_early_releases_table.php
├── 000031_add_missed_period_to_early_releases.php
├── 000032_refactor_early_releases_flow.php
├── 000034_add_classroom_id_to_users_table.php
├── 000040_create_audit_logs_table.php
├── 000050_create_late_entries_table.php
├── 000051_add_missed_period_to_late_entries.php
├── 000060_change_missed_period_to_json_in_releases_tables.php
├── seeders/
│   └── DatabaseSeeder.php                # Dados de teste (2 turmas, 4
├── lang/
│   └── pt_BR/
│       ├── validation.php                # Mensagens de validação em po
│       ├── auth.php
│       ├── pagination.php
│       └── passwords.php
├── resources/
│   ├── views/
│   │   ├── layouts/
│   │   │   └── app.blade.php            # Layout principal com sidebar
│   │   ├── dashboard/
│   │   │   ├── coordinator.blade.php
│   │   │   ├── teacher.blade.php
│   │   │   └── gatekeeper.blade.php
│   │   ├── early-releases/
│   │   │   ├── index.blade.php          # Listagem com filtros
│   │   │   ├── create.blade.php         # Formulário + busca dinâmica
│   │   │   └── show.blade.php           # Detalhes + linha do tempo 3
│   │   ├── late-entries/
│   │   │   ├── index.blade.php
│   │   │   ├── create.blade.php
│   │   │   └── show.blade.php           # Detalhes + linha do tempo 2
│   │   ├── students/
│   │   │   ├── index.blade.php
│   │   │   ├── create.blade.php
│   │   │   ├── edit.blade.php
│   │   │   └── show.blade.php           # Histórico completo do aluno
│   │   ├── classrooms/
│   │   │   ├── index.blade.php
│   │   │   ├── create.blade.php
│   │   │   └── edit.blade.php
│   │   ├── users/
│   │   │   ├── index.blade.php          # Lista com coluna de turma
│   │   │   ├── create.blade.php         # Campo turma aparece se profe
│   │   │   └── edit.blade.php
│   │   ├── reports/
│   │   │   └── index.blade.php
│   │   └── audit-logs/
│   │       └── index.blade.php

```

```
├── components/
│   ├── status-badge.blade.php      # Badge para early_release
│   └── late-status-badge.blade.php  # Badge para late_entries
└── css/app.css

├── routes/
│   ├── web.php                    # Todas as rotas da aplicação
│   └── auth.php                  # Rotas de autenticação (Breeze)
```

16. Regras de Negócio

Vínculo Professor ↔ Turma

- Cada professor pode ser vinculado a **no máximo uma turma** (`classroom_id` na tabela `users`)
- Cada turma pode ter **no máximo um professor** responsável (relação `HasOne`)
- Professor sem turma vinculada: **não visualiza nenhum registro**
- O badge de pendências na sidebar do professor conta apenas registros da **sua turma**
- A vinculação é feita pelo coordenador na criação ou edição do usuário

Controle de Duplicidade

- Um aluno **não pode ter dois registros de saída pendentes simultaneamente** (status `waiting_teacher` ou `waiting_gate`)
- Um aluno **não pode ter dois registros de entrada atrasada pendentes simultaneamente** (status `waiting_teacher`)
- O formulário de busca de alunos indica visualmente se o aluno possui registro pendente

Exclusão Lógica

- Alunos e usuários **nunca são deletados fisicamente** — o campo `active` é definido como `false`
- Turmas também seguem o mesmo padrão (campo `active`)
- Registros históricos de saídas/entradas são preservados mesmo após desativação

Aulas Perdidas (`missed_periods`)

- Armazenadas como **array JSON** (ex: `[ 2, 3 ]`)
- Seleção múltipla obrigatória (ao menos 1 período)
- Valores válidos: inteiros de 1 a 5
- Exibidas como badges coloridos nas listagens e detalhes

Timezone

- Todos os timestamps são gravados e exibidos no fuso horário `America/Sao_Paulo` (UTC-3)
- Formato de exibição: `d/m/Y H:i` (ex: `27/05/2026 14:30`)

Segurança e Autorização

- Toda rota protegida por middleware `auth` + `verified`
- Rotas adicionalmente protegidas por `RoleMiddleware` nos grupos de rotas
- Cada ação sensível verificada por **Policy** (dupla camada de segurança)
- Tokens CSRF em todos os formulários de estado
- Senhas com bcrypt (12 rounds)